

design better systems.

Software Architecture Patterns Explained

When to use them—and when not to.

What is Software Architecture?

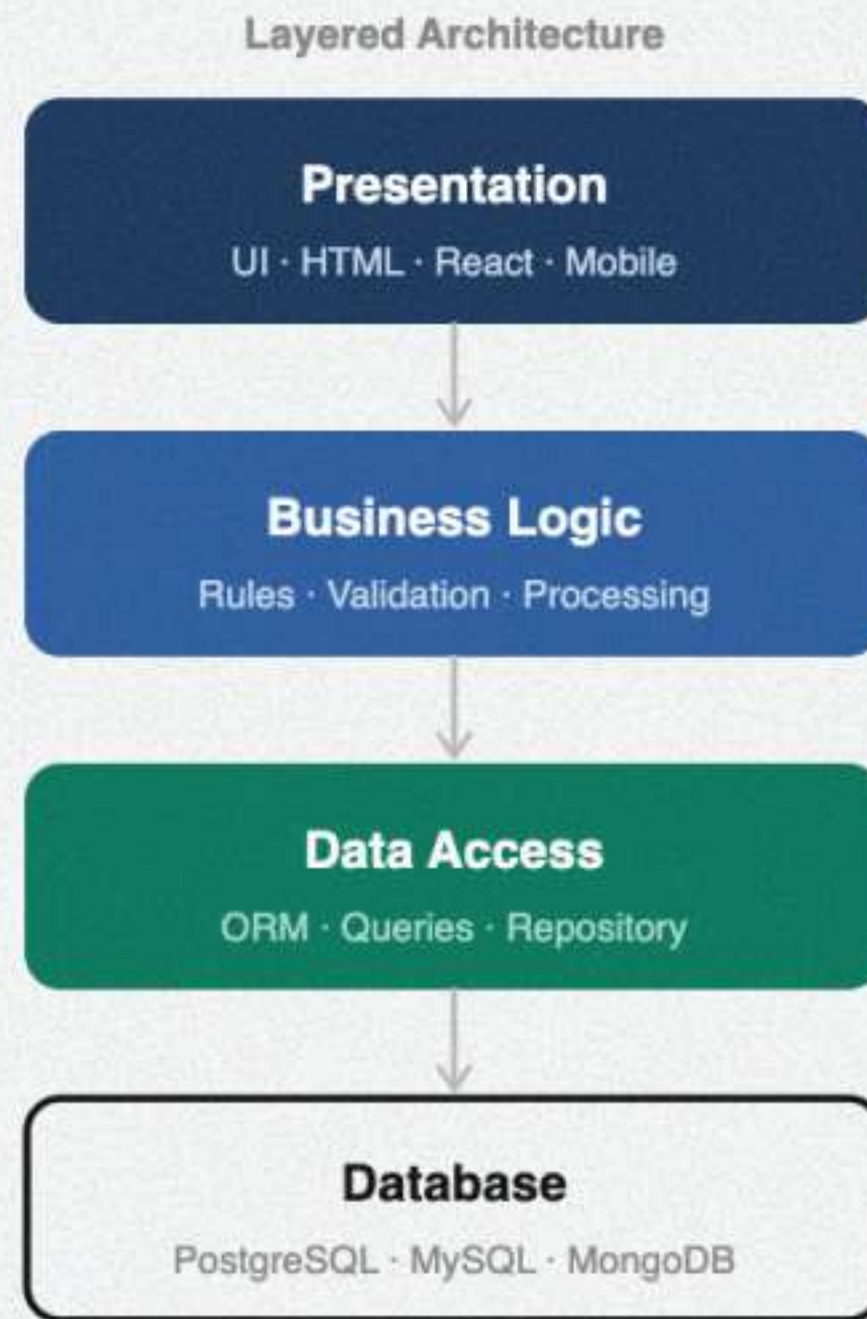
Software architecture is the **high-level structure** of an application.

It defines:

- How components are organised
- How they communicate
- Where responsibilities belong
- How the system can scale and evolve

Think of it as the blueprint before writing the code.

1. Layered Architecture



Best for

- **CRUD applications**
- **Admin dashboards**
- **Internal tools**
- **Most business applications**

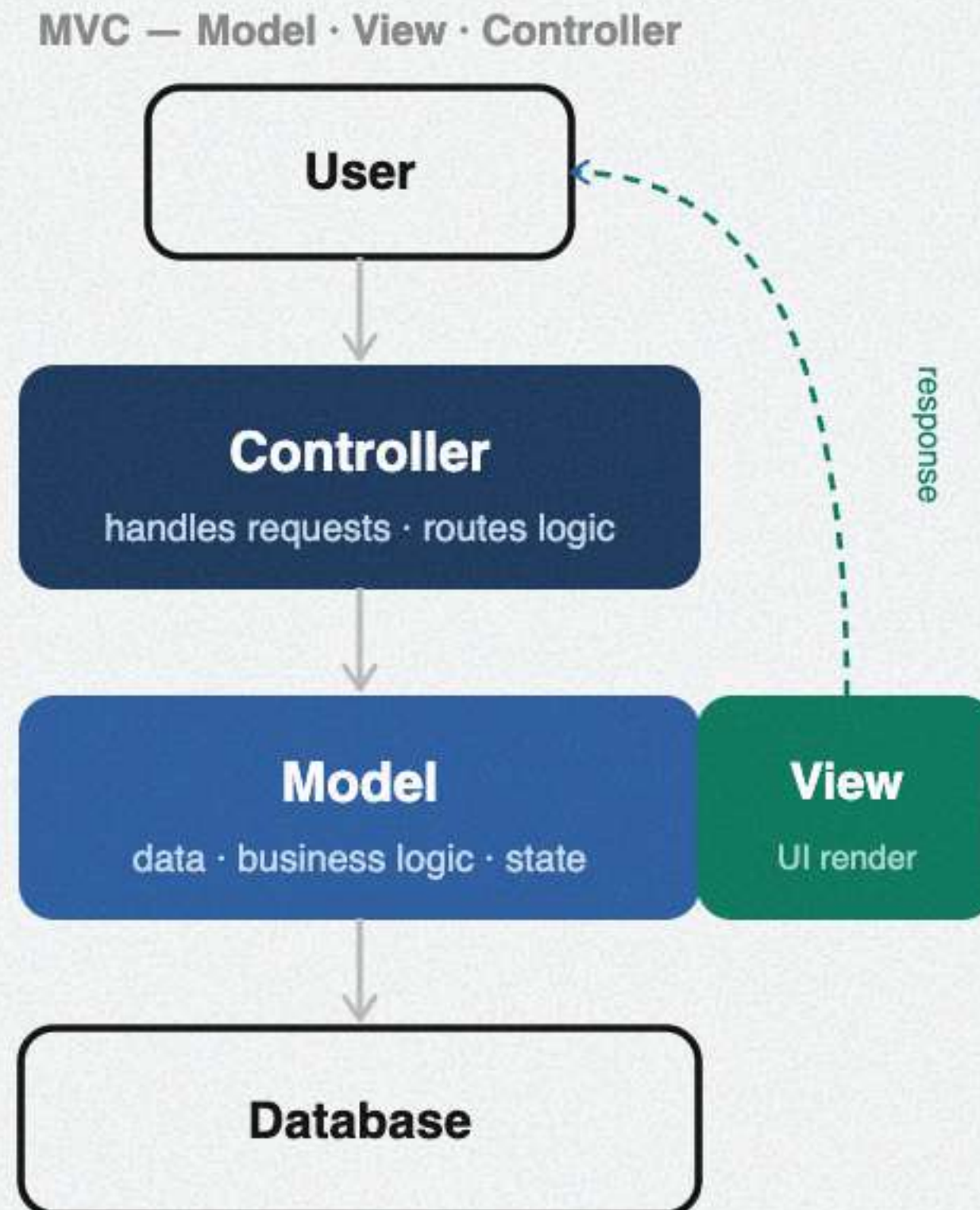
Advantages

- ✓ **Easy to understand**
- ✓ **Simple to maintain**
- ✓ **Great for beginners**

Trade-off

As projects grow, layers can become tightly coupled and harder to change independently.

2. MVC (Model–View–Controller)



Best for

- Web applications
- Server-rendered apps
- Frameworks like Laravel, Rails, ASP.NET MVC

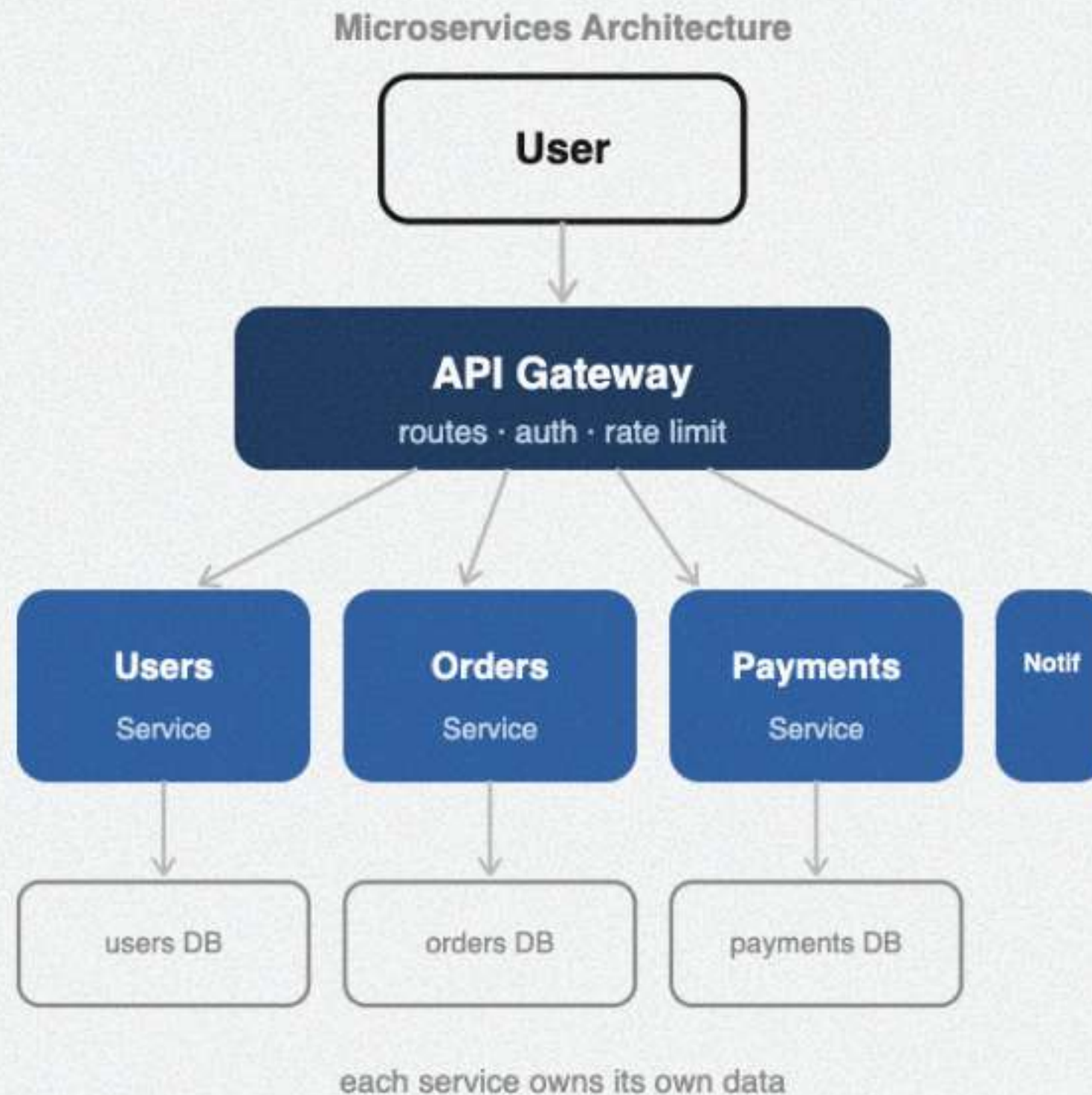
Advantages

- ✓ Clear separation of responsibilities
- ✓ Eases testing and maintenance

Trade-off

Can become difficult to manage if business logic starts leaking into controllers.

3. Microservices



Best for

- Large engineering teams
- Independent deployments
- Complex business domains

Advantages

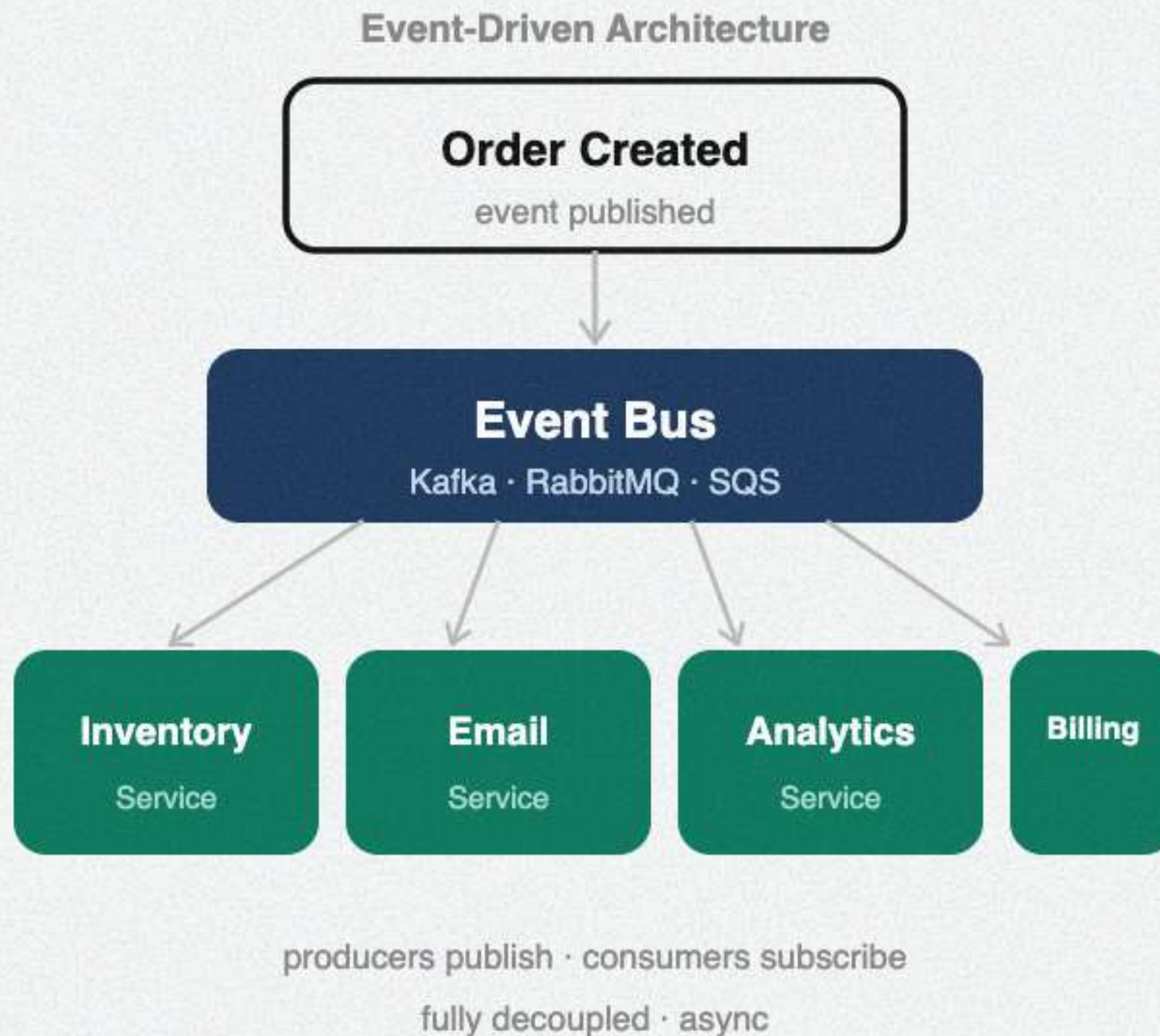
- ✓ Independent scaling
- ✓ Independent deployments
- ✓ Fault isolation

Trade-off

Adds operational complexity, networking challenges, and distributed system concerns.

Don't choose microservices just because large companies use them.

4. Event-Driven Architecture



Best for

- Notifications
- Messaging systems
- Real-time applications
- Background processing

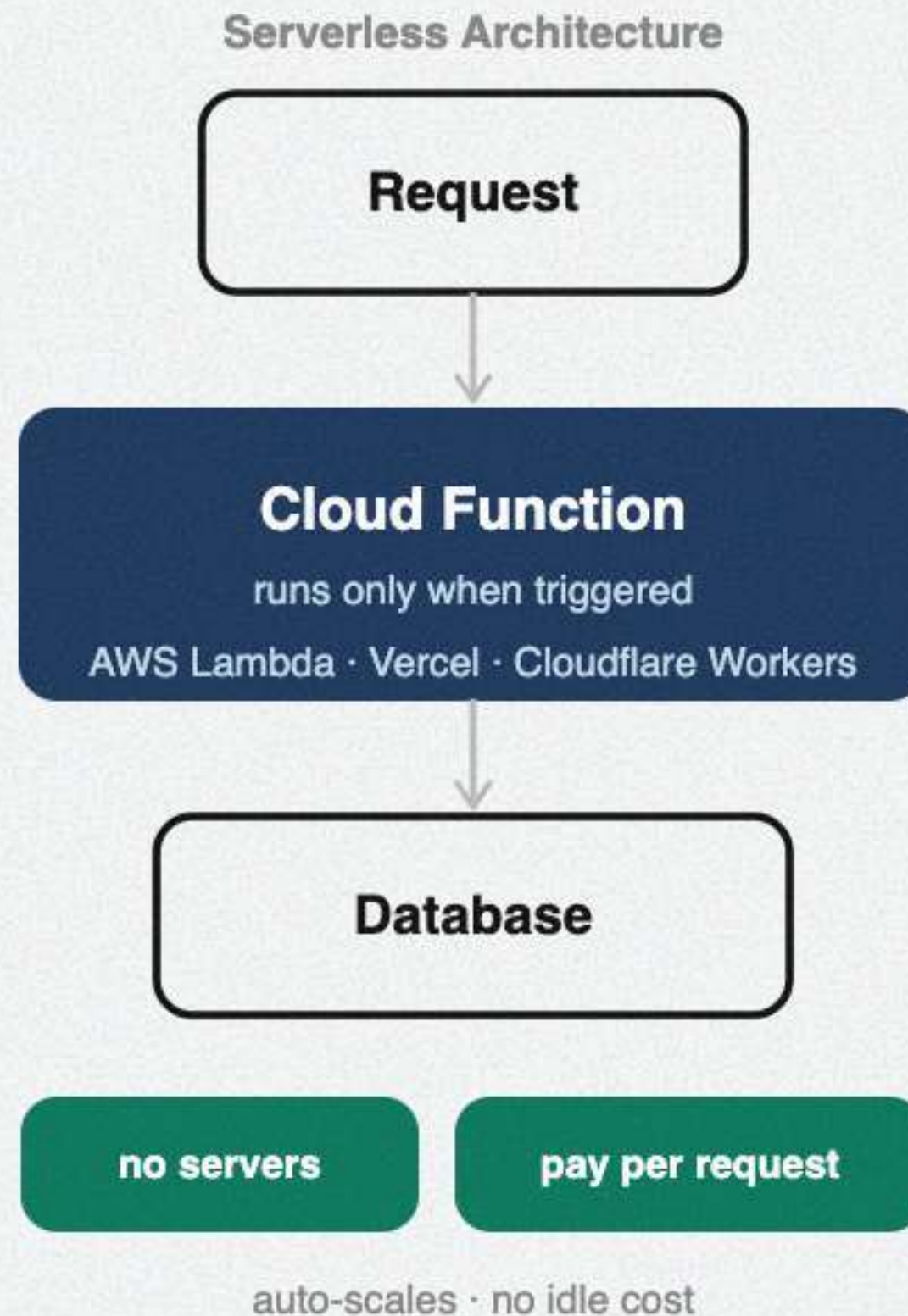
Advantages

- ✓ Loose coupling
- ✓ Easy to add new consumers
- ✓ Highly scalable

Trade-off

Debugging event flows can be more difficult than direct request-response systems.

5. Serverless



Best for

- APIs
- Automation
- Scheduled jobs
- Prototypes

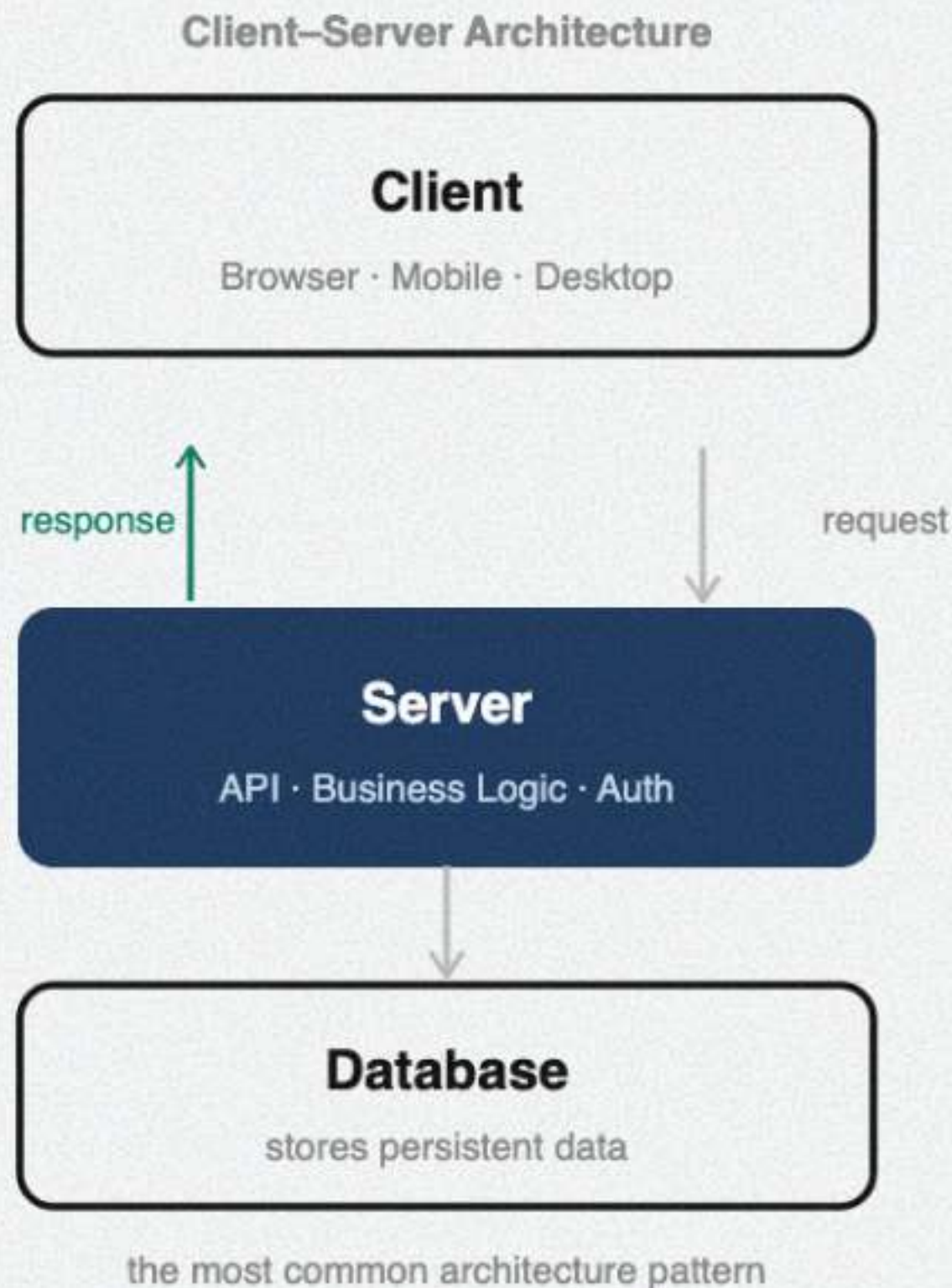
Advantages

- ✓ No server management
- ✓ Automatic scaling
- ✓ Pay for usage

Trade-off

Cold starts, vendor-specific services, and execution limits may affect some workloads.

6. Client–Server



Best for

- Most web applications
- Mobile apps
- Desktop clients
- APIs

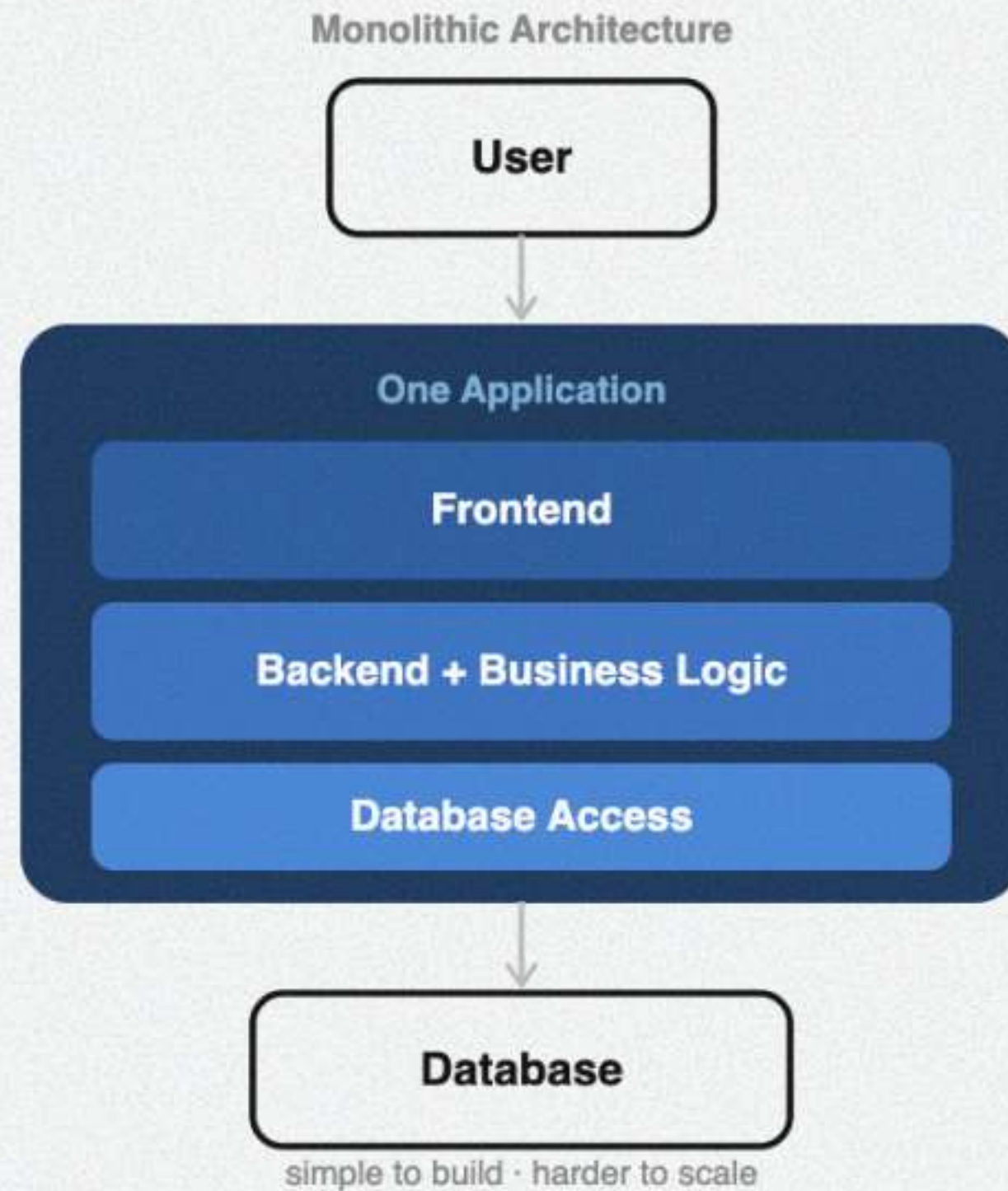
Advantages

- ✓ Simple architecture
- ✓ Easy communication
- ✓ Widely used

Trade-off

The server can become a bottleneck if it isn't designed to scale.

7. Monolithic Architecture



Best for

- MVPs
- Startups
- Small teams
- Products built by a few developers

Advantages

- ✓ Fast development
- ✓ Easy deployment
- ✓ Simpler debugging

Trade-off

As the application grows, deployments and scaling become more challenging.

Which Pattern Should You Choose?

If you're building...	Consider...
Startup MVP	Monolith
Admin Dashboard	Layered
Web App	MVC
Enterprise Product	Microservices
Notification System	Event-Driven
Small APIs	Serverless

There isn't a "best" architecture.

There is only the architecture that best fits your problem.

**Great software isn't defined by the
architecture it uses.**

**It's defined by whether the
architecture helps solve the
problem effectively.**

**Understand the strengths and
trade-offs of each pattern before
choosing one.**

 **Save this guide for future
reference.**

 **Comment "ARCHITECTURE" if this
helped simplify these concepts.**